



Universidad Tecnológica de Querétaro

INGENIERÍA EN DESARROLLO Y GESTIÓN DE SOFTWARE

Materia:

Seguridad en el Desarrollo de Aplicaciones

Maestro:

Gerardo Ramírez Villarreal

Grupo:

IDGS17

Nombre Alumno:

Luis Eduardo Rios Cervantes

Trabajo:

Cross-Site Scripting XSS

Querétaro, Qro. 04 de Junio de 2026

Ingresar un comentario simple.

[VulnerableApp](#) 

- [Home](#)
- [Buscar Usuarios](#)
- [Login](#)
- [Componente de comentarios](#)
- [Privacy](#)

Comentarios

Primer comentario



Comentar

© 2026 - VulnerableApp - [Privacy](#)

Verificar que se muestre en pantalla.

[VulnerableApp](#) 

- [Home](#)
- [Buscar Usuarios](#)
- [Login](#)
- [Componente de comentarios](#)
- [Privacy](#)

Comentarios

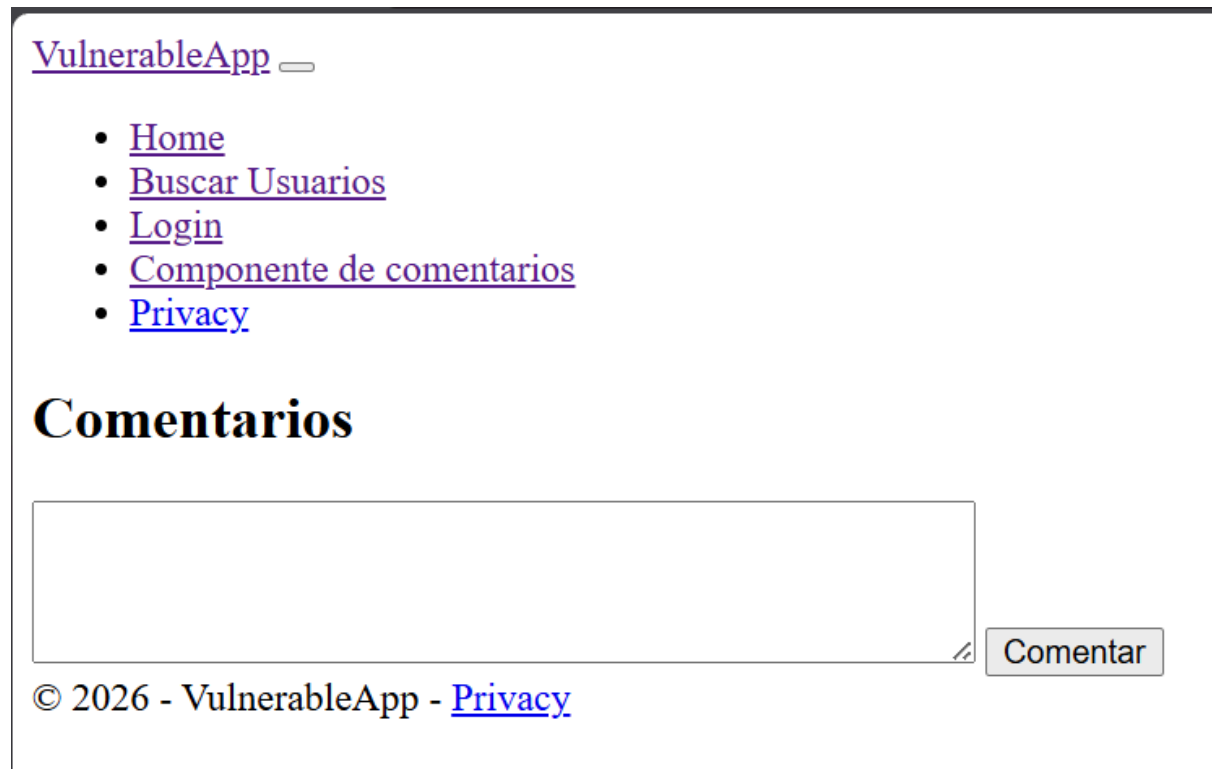


Comentar

Primer comentario

© 2026 - VulnerableApp - [Privacy](#)

Reiniciar la aplicación y observar el comportamiento de la lista estática.

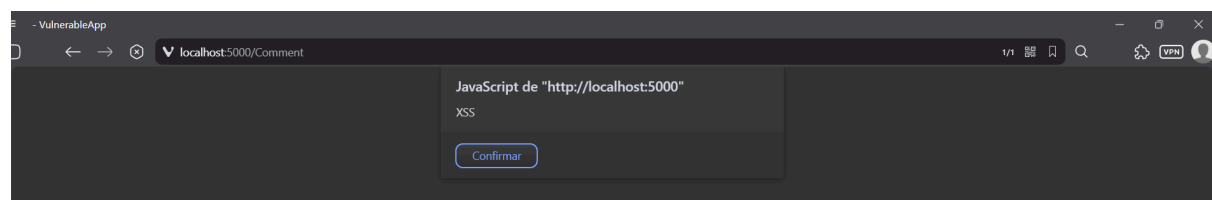


Al reiniciar la aplicación, la lista de comentarios desaparece, debido a que esta lista es estática, solo se almacena en el caché de la aplicación.

Captura de contenido no confiable ingresado.



Captura del código vulnerable.



¿Por qué el navegador interpreta el contenido?

Porque el navegador web no procesa adecuadamente lo que el usuario ingresa (en este caso, un texto que contiene etiquetas `<script>`) y simplemente lo inserta en el código fuente de la página y lo envía al navegador. El navegador no puede distinguir entre un script programado por los desarrolladores y el script inyectado por un tercero, por lo que procede a ejecutarlo directamente.

Identificar qué información podría estar en riesgo.

Cookies de sesión, Datos almacenados en el navegador, Información ingresada por el usuario, Tokens de autenticación.

Relacionar el hallazgo con validación de entrada y codificación de salida.

`<script>alert('XSS')</script>` demuestra una vulnerabilidad de Cross-Site Scripting (XSS). Representa un fallo grave donde la aplicación web permite la inyección y ejecución arbitraria de código JavaScript en el navegador de los usuarios. Para prevenir ataques XSS, debemos implementar validaciones en los inputs, como que sea solo numérico o solo de fechas, y agregar formateadores de texto. Para la codificación de salida, necesitamos convertir caracteres especiales (como `<`, `>`, `"`, `'`, y `/`) en sus equivalentes de entidad HTML seguros (como `<` y `>`) antes de que el navegador los muestre en pantalla.